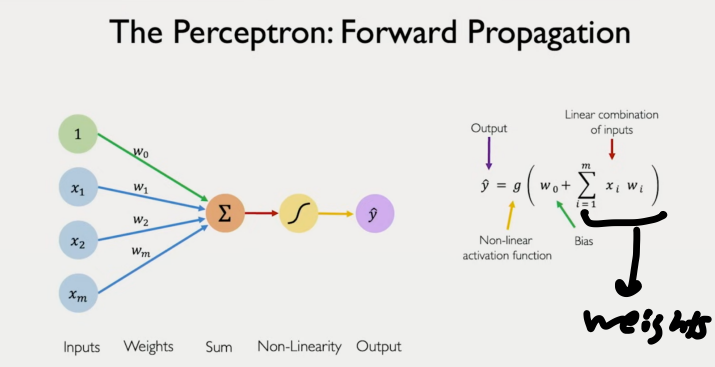
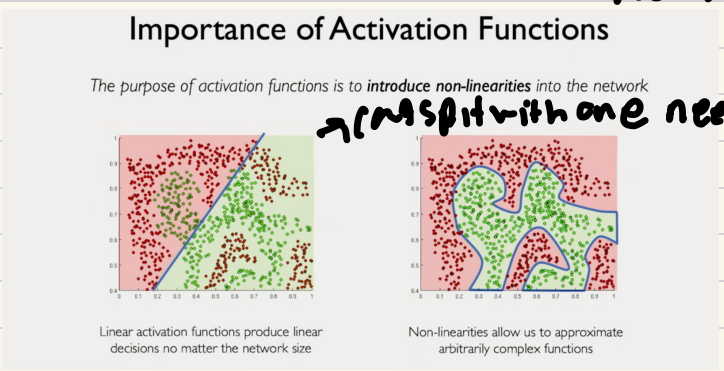
AI Deep Learning MIT 6.S191
Basic Deep Learning Video → 15k USD compute → Advances
Cool cloning voice Example: 6:28
Deep Learning: Extract patterns from data using neural networks
M.L. = Ability to learn without explicitly being programmed] subset ←
Deep Learning
made from features
Structure of Neural Network (Deep Learning
Perceptron
↳ Basic unit of information
↳ multiply by weights
→ Bias term
↳ Shift activation function

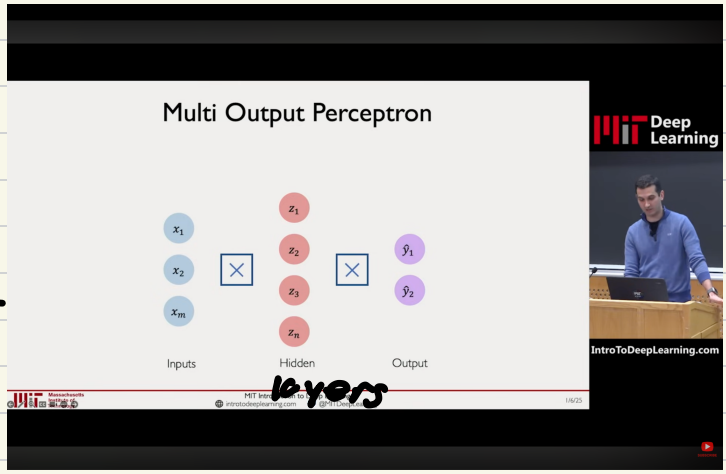


Sigmoid function → fit btwn 0, 1
↳ Activation function: Introduce Non-linearity
Real data heavily non-linear



→ (NN split with one needs non-linearities

Neural Network with Perceptrons
Multi-Output NN
↳ Two Neurons
↳ different weight
↳ Different output



Dense Layers: Everything input connects output
Singe layer: Not from one Input
↳ Hidden
↳ Can't observe data
↳ Learned over observed data

Deep Neural Network
Complex Tasks → More Dense Layers
Applying NN

In order To Train, model understand when make bad Predictions
Loss: From predictions to Actual values, small is closer to predicted (Actual)] Residual
Empirical Loss: Training on entire dataset, average of loss for entire dataset
MSE Loss → Numerical
Binary Loss → Yes/No
↗ Soft-Max Cross Entropy Loss

Training Neural Network
- Finding weights
  - ↳ Minimize Loss

We want to find the network weights that **achieve the lowest loss**

$$W^* = \underset{W}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(f\left(x^{(i)}; W\right), y^{(i)}\right)$$

$$W^* = \underset{W}{\operatorname{argmin}} J(W)$$

L = Loss
J → weights
] minimize loss through weights

Loss Optimization
Gradient → which way is up, with Loss
→ go opposite go down with Loss
Gradient Descent
- Randomly pick weights
- Compute gradient
  - ↳ Direction to inc loss
  - ↳ Go in other direction

Update weight by multiply by step to show how much to go down

Gradient
Back-Propagation = Chain Rule
↳ One for every weight
↳ Tells if inc loss
↳ opposite direction
↳ Tells if dec loss
↳ Usually inc weight

Step → Learning Rate
↳ How fast take steps
optimize LR to not
miss Local Minima
Stochastic Gradient Descent
- Randomly pick one data point
- Quicker than normal (gradient descent

Trust gradient more through optimization

Overfitting
↳ Leverage training pts but not memorizing it
↳ Regularization: Discourage memorization
↳ Dropout
↳ Randomly set neurons to 0
↳ Forces network to not rely on at pts on any wine neuron
↳ Early Stopping
↳ Mon. for Deviation From Training Loss Including Set
↳ Testing / Training
30 ↓ 70